

Working with lists

Tad Dallas

Contents

apply statements	10
plyr apply functionality tweaks	13
sessionInfo	20

What are lists?

Lists in R are collections of objects that can be of any mix of types (or all the same type). They can be useful when dealing with multiple data.frames that each correspond to a different unit of study (note that we solved this before by considering a single data.frame with a column corresponding to country). Lists tend to be useful in my research when I'm simulating ecological dynamics, where each list item can be the results of a single simulation, or when I'm writing functions to return data that is not really structured as a data.frame. Before we go too far into use cases, let's refresh on how we form and index list objects.

```
lst <- list(a=runif(100), df=data.frame(x=runif(100), y=runif(100)), letters[1:10], 'a')
```

```
# index single list elements
```

```
lst[[1]]
```

```
##      [1] 0.454684807 0.828998899 0.850822154 0.449602484 0.552855304 0.945385263
##      [7] 0.817799580 0.182763191 0.739489115 0.780848301 0.296204605 0.466535769
##     [13] 0.448727593 0.833433425 0.689203075 0.722213078 0.491429181 0.242051459
##     [19] 0.914824286 0.087667044 0.784766432 0.757817168 0.803616947 0.784012350
##     [25] 0.056701507 0.920384082 0.388655300 0.584912032 0.706854938 0.246418147
##     [31] 0.291095118 0.032800708 0.113504986 0.644342415 0.871213629 0.560752011
##     [37] 0.787274353 0.189651222 0.241549481 0.875186021 0.305097832 0.228675810
##     [43] 0.121290770 0.172697602 0.628219830 0.753972096 0.352425098 0.812702204
##     [49] 0.343629715 0.427234220 0.135826056 0.360087817 0.657891924 0.591383324
##     [55] 0.315930174 0.086641267 0.460036133 0.828720973 0.152162513 0.919483991
##     [61] 0.305608512 0.858496239 0.968245148 0.640975363 0.172007313 0.159768183
##     [67] 0.462653191 0.185213919 0.512710287 0.892515375 0.547172626 0.122429744
##     [73] 0.158959549 0.373213555 0.806175020 0.746117110 0.749345450 0.613794070
##     [79] 0.183428118 0.837301489 0.151312279 0.792934709 0.569813581 0.234390738
##     [85] 0.769642195 0.679619507 0.163374105 0.560434017 0.430119107 0.891042639
##     [91] 0.873710843 0.866823527 0.004539114 0.353211500 0.547337207 0.500758941
##     [97] 0.641781396 0.531918178 0.679969502 0.840258856
```

```
lst[[2]]
```

```
##           x           y
## 1 0.8887521422 0.626542531
## 2 0.2383874054 0.811535648
## 3 0.8349729427 0.922685312
## 4 0.8601783121 0.319172522
## 5 0.2970715393 0.538666641
```

## 6	0.1436799874	0.314270010
## 7	0.6915531920	0.827500332
## 8	0.1128045898	0.401718416
## 9	0.2657445516	0.724670956
## 10	0.7603118299	0.661804271
## 11	0.6996424692	0.005507458
## 12	0.8340254822	0.183356077
## 13	0.9131521937	0.581927575
## 14	0.6537267026	0.986661190
## 15	0.6980947668	0.717332633
## 16	0.6365805725	0.934611890
## 17	0.0003171435	0.963897777
## 18	0.8466229427	0.338584602
## 19	0.6605226586	0.059817110
## 20	0.1352035371	0.521748853
## 21	0.0238553591	0.345447130
## 22	0.2286542368	0.453791636
## 23	0.6860492404	0.378277944
## 24	0.6590818826	0.636675431
## 25	0.6682495866	0.707493474
## 26	0.1951388055	0.832010234
## 27	0.4142748604	0.828289328
## 28	0.1784768216	0.157502300
## 29	0.2791445618	0.311826878
## 30	0.1439296072	0.841177171
## 31	0.0314848037	0.107277608
## 32	0.4999301077	0.821242370
## 33	0.9495378444	0.375183064
## 34	0.4081737993	0.751485717
## 35	0.9444211589	0.314154578
## 36	0.9309000378	0.263778163
## 37	0.2730433585	0.677771977
## 38	0.2332939941	0.010945949
## 39	0.4947811353	0.612974369
## 40	0.9041969429	0.562410644
## 41	0.2278147833	0.214229320
## 42	0.4120053146	0.801170178
## 43	0.9649036727	0.459928802
## 44	0.5688418187	0.873139778
## 45	0.8787742588	0.319849428
## 46	0.2928586854	0.229381530
## 47	0.7365415352	0.839950873
## 48	0.0199826197	0.949850055
## 49	0.5405890294	0.487463669
## 50	0.3518576145	0.940629345
## 51	0.1186117455	0.860961390
## 52	0.4906090992	0.046118343
## 53	0.0915425876	0.122408253
## 54	0.6995467143	0.519082003
## 55	0.0151303415	0.202532387
## 56	0.0316851439	0.171893855
## 57	0.0828541226	0.236695342
## 58	0.6521523451	0.188689979
## 59	0.5817908973	0.021467544

```
## 60 0.1359395208 0.446979143
## 61 0.0383476201 0.544126714
## 62 0.9397749207 0.708145951
## 63 0.2099468696 0.566083273
## 64 0.1092199378 0.657214117
## 65 0.8909011758 0.610636319
## 66 0.3471619557 0.863716759
## 67 0.3368265897 0.114855236
## 68 0.1142973234 0.434243454
## 69 0.7565642339 0.263416885
## 70 0.2239717059 0.176563327
## 71 0.4238559573 0.524589761
## 72 0.0361162981 0.861914084
## 73 0.1440987822 0.466847110
## 74 0.6797856605 0.074147722
## 75 0.7070562679 0.518575839
## 76 0.0521127509 0.828331620
## 77 0.6621992139 0.548056602
## 78 0.6158492467 0.306321634
## 79 0.5302105148 0.007326476
## 80 0.6300003284 0.927355272
## 81 0.1043345067 0.156847954
## 82 0.6511174864 0.372755819
## 83 0.1430475402 0.485551410
## 84 0.8825128225 0.650498784
## 85 0.9853099366 0.391652576
## 86 0.6399432130 0.295233517
## 87 0.9427770283 0.596842784
## 88 0.1802212216 0.868911036
## 89 0.9192820212 0.960700932
## 90 0.2279071321 0.803493483
## 91 0.3850981961 0.418654700
## 92 0.9792314728 0.342356649
## 93 0.1626621082 0.222663791
## 94 0.9192913459 0.654100325
## 95 0.5302369997 0.412861278
## 96 0.3278196421 0.614841274
## 97 0.2180372248 0.060191197
## 98 0.6331381232 0.439606141
## 99 0.9850425236 0.179171206
## 100 0.3608595266 0.879314804
```

```
lst[[3]]
```

```
## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```
lst[[4]]
```

```
## [1] "a"
```

```
# index multiple list elements
```

```
lst[1:2]
```

```
## $a
```

```
## [1] 0.454684807 0.828998899 0.850822154 0.449602484 0.552855304 0.945385263
## [7] 0.817799580 0.182763191 0.739489115 0.780848301 0.296204605 0.466535769
## [13] 0.448727593 0.833433425 0.689203075 0.722213078 0.491429181 0.242051459
```

```

## [19] 0.914824286 0.087667044 0.784766432 0.757817168 0.803616947 0.784012350
## [25] 0.056701507 0.920384082 0.388655300 0.584912032 0.706854938 0.246418147
## [31] 0.291095118 0.032800708 0.113504986 0.644342415 0.871213629 0.560752011
## [37] 0.787274353 0.189651222 0.241549481 0.875186021 0.305097832 0.228675810
## [43] 0.121290770 0.172697602 0.628219830 0.753972096 0.352425098 0.812702204
## [49] 0.343629715 0.427234220 0.135826056 0.360087817 0.657891924 0.591383324
## [55] 0.315930174 0.086641267 0.460036133 0.828720973 0.152162513 0.919483991
## [61] 0.305608512 0.858496239 0.968245148 0.640975363 0.172007313 0.159768183
## [67] 0.462653191 0.185213919 0.512710287 0.892515375 0.547172626 0.122429744
## [73] 0.158959549 0.373213555 0.806175020 0.746117110 0.749345450 0.613794070
## [79] 0.183428118 0.837301489 0.151312279 0.792934709 0.569813581 0.234390738
## [85] 0.769642195 0.679619507 0.163374105 0.560434017 0.430119107 0.891042639
## [91] 0.873710843 0.866823527 0.004539114 0.353211500 0.547337207 0.500758941
## [97] 0.641781396 0.531918178 0.679969502 0.840258856
##
## $df
##           x           y
## 1  0.8887521422 0.626542531
## 2  0.2383874054 0.811535648
## 3  0.8349729427 0.922685312
## 4  0.8601783121 0.319172522
## 5  0.2970715393 0.538666641
## 6  0.1436799874 0.314270010
## 7  0.6915531920 0.827500332
## 8  0.1128045898 0.401718416
## 9  0.2657445516 0.724670956
## 10 0.7603118299 0.661804271
## 11 0.6996424692 0.005507458
## 12 0.8340254822 0.183356077
## 13 0.9131521937 0.581927575
## 14 0.6537267026 0.986661190
## 15 0.6980947668 0.717332633
## 16 0.6365805725 0.934611890
## 17 0.0003171435 0.963897777
## 18 0.8466229427 0.338584602
## 19 0.6605226586 0.059817110
## 20 0.1352035371 0.521748853
## 21 0.0238553591 0.345447130
## 22 0.2286542368 0.453791636
## 23 0.6860492404 0.378277944
## 24 0.6590818826 0.636675431
## 25 0.6682495866 0.707493474
## 26 0.1951388055 0.832010234
## 27 0.4142748604 0.828289328
## 28 0.1784768216 0.157502300
## 29 0.2791445618 0.311826878
## 30 0.1439296072 0.841177171
## 31 0.0314848037 0.107277608
## 32 0.4999301077 0.821242370
## 33 0.9495378444 0.375183064
## 34 0.4081737993 0.751485717
## 35 0.9444211589 0.314154578
## 36 0.9309000378 0.263778163
## 37 0.2730433585 0.677771977

```

```

## 38 0.2332939941 0.010945949
## 39 0.4947811353 0.612974369
## 40 0.9041969429 0.562410644
## 41 0.2278147833 0.214229320
## 42 0.4120053146 0.801170178
## 43 0.9649036727 0.459928802
## 44 0.5688418187 0.873139778
## 45 0.8787742588 0.319849428
## 46 0.2928586854 0.229381530
## 47 0.7365415352 0.839950873
## 48 0.0199826197 0.949850055
## 49 0.5405890294 0.487463669
## 50 0.3518576145 0.940629345
## 51 0.1186117455 0.860961390
## 52 0.4906090992 0.046118343
## 53 0.0915425876 0.122408253
## 54 0.6995467143 0.519082003
## 55 0.0151303415 0.202532387
## 56 0.0316851439 0.171893855
## 57 0.0828541226 0.236695342
## 58 0.6521523451 0.188689979
## 59 0.5817908973 0.021467544
## 60 0.1359395208 0.446979143
## 61 0.0383476201 0.544126714
## 62 0.9397749207 0.708145951
## 63 0.2099468696 0.566083273
## 64 0.1092199378 0.657214117
## 65 0.8909011758 0.610636319
## 66 0.3471619557 0.863716759
## 67 0.3368265897 0.114855236
## 68 0.1142973234 0.434243454
## 69 0.7565642339 0.263416885
## 70 0.2239717059 0.176563327
## 71 0.4238559573 0.524589761
## 72 0.0361162981 0.861914084
## 73 0.1440987822 0.466847110
## 74 0.6797856605 0.074147722
## 75 0.7070562679 0.518575839
## 76 0.0521127509 0.828331620
## 77 0.6621992139 0.548056602
## 78 0.6158492467 0.306321634
## 79 0.5302105148 0.007326476
## 80 0.6300003284 0.927355272
## 81 0.1043345067 0.156847954
## 82 0.6511174864 0.372755819
## 83 0.1430475402 0.485551410
## 84 0.8825128225 0.650498784
## 85 0.9853099366 0.391652576
## 86 0.6399432130 0.295233517
## 87 0.9427770283 0.596842784
## 88 0.1802212216 0.868911036
## 89 0.9192820212 0.960700932
## 90 0.2279071321 0.803493483
## 91 0.3850981961 0.418654700

```

```
## 92 0.9792314728 0.342356649
## 93 0.1626621082 0.222663791
## 94 0.9192913459 0.654100325
## 95 0.5302369997 0.412861278
## 96 0.3278196421 0.614841274
## 97 0.2180372248 0.060191197
## 98 0.6331381232 0.439606141
## 99 0.9850425236 0.179171206
## 100 0.3608595266 0.879314804
```

Let's think about how we might use lists. For one, many functions in R output data in list format. For instance, working with network data in R through **igraph**, most things are lists. Let's explore this, both as a way to introduce lists and to talk about how to analyze network data in R.

```
#install.packages('igraph')
library(igraph)

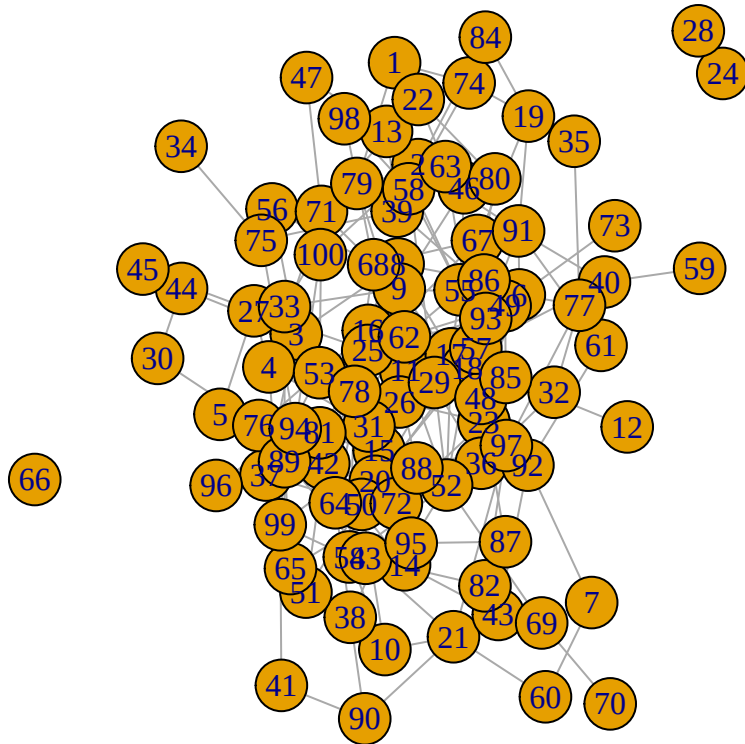
g <- igraph::sample_gnm(100, 200)

str(g)
```

```
## Class 'igraph'  hidden list of 10
## $ : num 100
## $ : logi FALSE
## $ : num [1:200] 7 8 8 12 15 16 16 16 19 20 ...
## $ : num [1:200] 2 5 7 1 1 5 7 14 17 9 ...
## $ : NULL
## $ : NULL
## $ : NULL
## $ : NULL
## $ :List of 4
## ..$ : num [1:3] 1 0 1
## ..$ :List of 4
## .. ..$ name : chr "Erdos-Renyi (gnm) graph"
## .. ..$ type : chr "gnm"
## .. ..$ loops: logi FALSE
## .. ..$ m : num 200
## ..$ : Named list()
## ..$ : Named list()
## $ :<environment: 0x5a3d94531d28>
```

Recall when we introduced the `plot` function, and I said that packages build in functionality such that some base functions will work with more complex objects (the **igraph** object above is a list). Try it here.

```
plot(g)
```

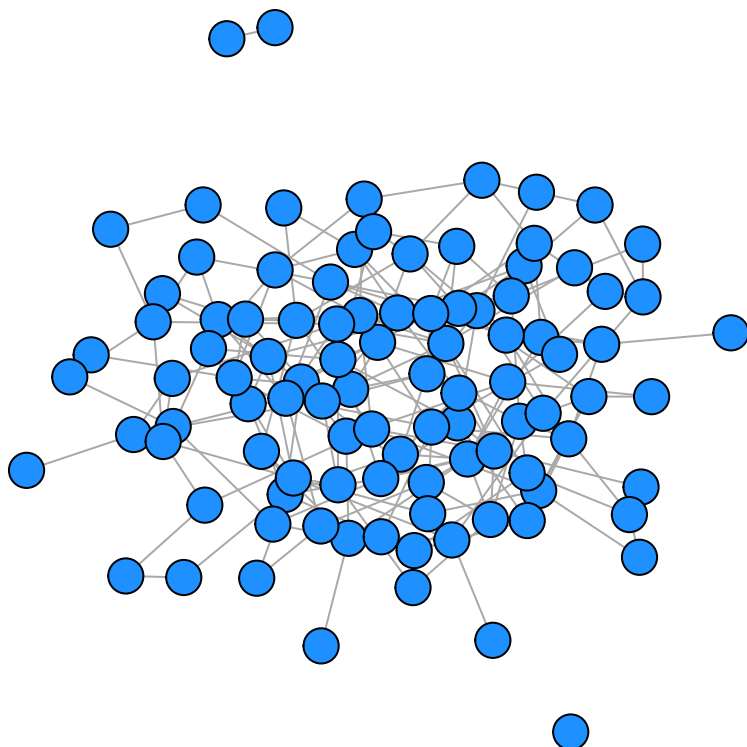


Nice. That's neat. We can also write a wrapper function (we will not go over function writing now, but will save that for the coming weeks), which can be useful across multiple projects. This is a function I routinely use for visualizing networks in a prettier way.

```
## @param g graph object
## @param colz
## @param nodeColor
## @param nodeSize
##
## @return a graph plot

plotGraph <- function(g, lay=layout_nicely(g), colz='dodgerblue'){
  plot(g, layout=lay, edge.width= E(g)$weight, vertex.size=10, directed=FALSE,
       vertex.color= colz,
       vertex.label=NA)
}

plotGraph(g)
```



But let's get back to lists. We've seen that **igraph** graph objects are lists, but also many of the outputs of functions within **igraph** are lists (or even lists of lists!). I will not defend nested lists as being all that useful, but we will see them in a couple of weeks when we talk about APIs and spatial data.

So one of the functions built into **igraph** is the **sir** function. This is a function which runs a model on the network known as the Susceptible-Infected-Recovered model (or SIR for short), which aims to capture how diseases spread through populations.

$$\frac{dS}{dt} = -\beta SI \quad (1)$$

$$\frac{dI}{dt} = \beta SI - \gamma I \quad (2)$$

$$\frac{dR}{dt} = \gamma I \quad (3)$$

The default behavior of the function (**?sir**) will run 100 simulations of the SIR model on a graph object that you provide to the function, and store the output as a list.

```
sims <- igraph::sir(g, beta=0.5, gamma=0.5, no.sim=100)
typeof(sims)
```

```
## [1] "list"
```

```
class(sims)
```

```
## [1] "sir"
```

```
# explore the structure of each list element
sims[[1]]
```

```
## $times
```

```
## [1] 0.000000 1.043111
```



```
##
## $NS
## [1] 99 99
##
## $NI
## [1] 1 0
##
## $NR
## [1] 0 1
# each list element is another list
sims[[1]][[1]]
```

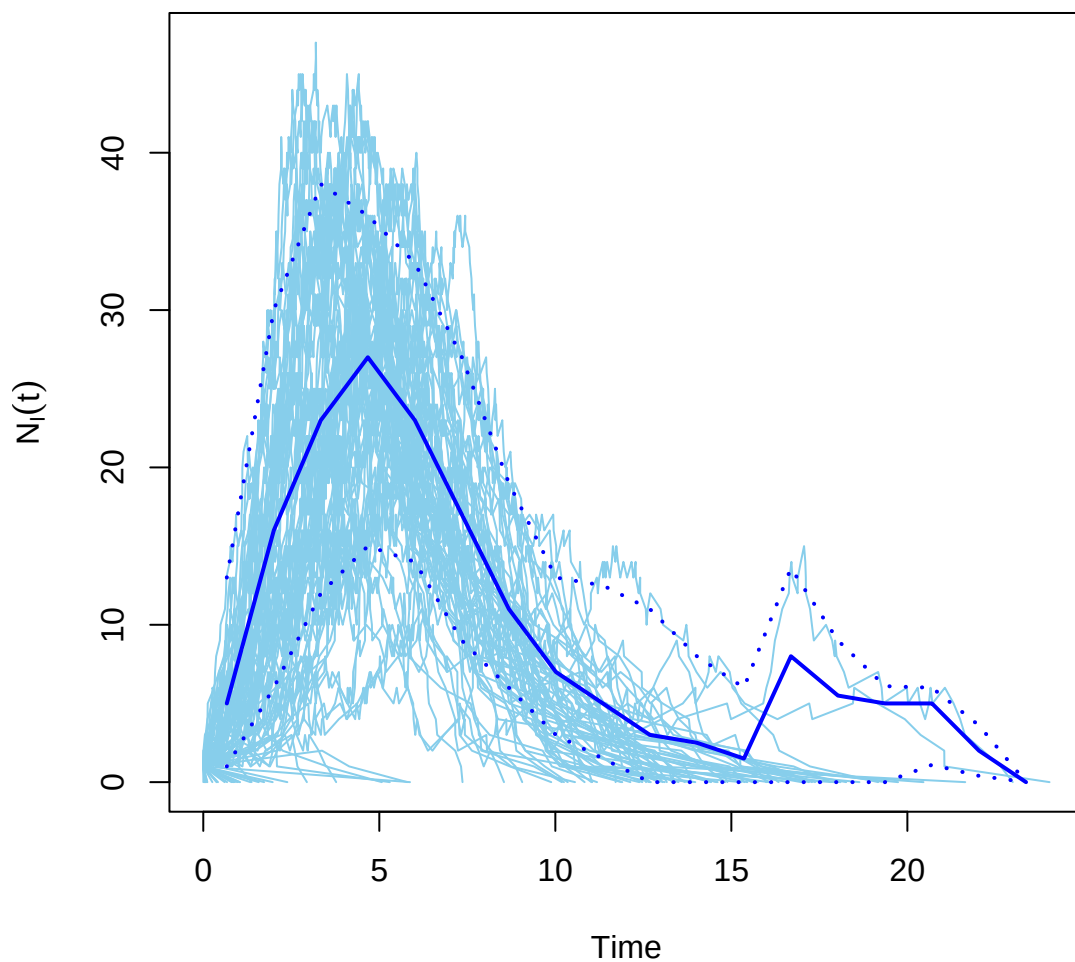
```
## [1] 0.000000 1.043111
```

```
sims[[1]]$times
```

```
## [1] 0.000000 1.043111
```

And just to go back to plotting for a quick second, `igraph` has written functionality into the `sir` class to work well with the base R `plot` function.

```
plot(sims)
```



Neat, right?

But back to lists. Let's work through the rest of working with lists through some exercises. Given the

simulations above (`sims` list) ...

Calculate the mean number of infected individuals for each simulation

Find the time associated with the maximum number of infected nodes

Calculate the fraction of all simulations in which fewer than 5 nodes are infected

apply statements

How did we approach the above questions? You almost certainly used a `for` loop, right? This is perfectly fine, but there is another way. `apply` statements allow you to take a function and run it over all elements of a vector, columns/rows of a matrix, or a list.

`apply` statements typically have a prefix which gives information about what type of data it works well with. For instance, working with lists, we will use `lapply`. The base `apply` function is to work with matrices, where we want to apply a function to every row or column of a matrix (e.g., `apply(matrix, 2, sum)` is the same as `colSums(matrix)`). We will go over more examples of `apply` statements at some point, but for now we will focus on `lapply` and our problem of working with lists.

And here we hit an issue. `lapply` statements take a function argument, where the function needs to take the list object as an argument and then does something with it. So we'll have to learn a bare minimum of function writing now. Let's use a problem above as a motivating example, where we try to calculate the mean number of infected individuals for each simulation.

```
meanInfections <- function(x){  
  # if we consider the mean infecteds as the mean of the infected vector  
  return(mean(x$NI))  
  # if we consider the mean infecteds as the mean number of nodes infected  
  # return((x$NS[1]+1)-tail(x$NS,1))  
}  
  
meanInfs <- lapply(sims, meanInfections)  
str(meanInfs)
```

```
## List of 100  
## $ : num 0.5  
## $ : num 16.4  
## $ : num 12.6  
## $ : num 16.9  
## $ : num 27.2  
## $ : num 21.4  
## $ : num 22  
## $ : num 19  
## $ : num 15.2  
## $ : num 20.3  
## $ : num 20.9  
## $ : num 26  
## $ : num 24.1  
## $ : num 0.5  
## $ : num 0.5  
## $ : num 0.5  
## $ : num 0.5  
## $ : num 14.5  
## $ : num 20.2  
## $ : num 0.5
```

\$: num 22.4
\$: num 1
\$: num 0.5
\$: num 0.5
\$: num 14.1
\$: num 0.5
\$: num 3.07
\$: num 0.5
\$: num 12.6
\$: num 19.7
\$: num 0.5
\$: num 18.3
\$: num 24.4
\$: num 21.9
\$: num 16.5
\$: num 0.5
\$: num 17.1
\$: num 0.5
\$: num 1.5
\$: num 19
\$: num 18.2
\$: num 25.5
\$: num 2.38
\$: num 0.5
\$: num 12.9
\$: num 1
\$: num 19.1
\$: num 7.92
\$: num 22
\$: num 0.5
\$: num 24.6
\$: num 0.5
\$: num 0.5
\$: num 18.3
\$: num 24.9
\$: num 17.4
\$: num 0.5
\$: num 14.4
\$: num 0.5
\$: num 20.4
\$: num 19.1
\$: num 0.5
\$: num 25.7
\$: num 16.2
\$: num 15.1
\$: num 15.9
\$: num 12.6
\$: num 1
\$: num 26.9
\$: num 25.6
\$: num 19.4
\$: num 0.5
\$: num 21.3
\$: num 25.4

```
## $ : num 21.8
## $ : num 20.5
## $ : num 7.99
## $ : num 14.8
## $ : num 21.7
## $ : num 17.8
## $ : num 19.9
## $ : num 0.5
## $ : num 11.5
## $ : num 20.6
## $ : num 0.5
## $ : num 23.8
## $ : num 21
## $ : num 21.4
## $ : num 0.5
## $ : num 19.3
## $ : num 13.6
## $ : num 10.7
## $ : num 21.6
## $ : num 14
## $ : num 0.5
## $ : num 1
## $ : num 0.5
## $ : num 26.5
## $ : num 28.1
## [list output truncated]
```

`lapply` is nice in that if we give it a list object, it gives us a list object back. This makes analytical pipelines that deal with lists pretty straightforward, but if the output is a single value, we may want this to be a vector instead of a list.

```
unlist(meanInfs)
```

```
## [1] 0.500000 16.450000 12.551948 16.942857 27.151163 21.394118 21.960526
## [8] 19.021127 15.166667 20.265432 20.916667 26.038462 24.087500 0.500000
## [15] 0.500000 0.500000 0.500000 14.468750 20.186047 0.500000 22.432584
## [22] 1.000000 0.500000 0.500000 14.069620 0.500000 3.071429 0.500000
## [29] 12.644928 19.712500 0.500000 18.310127 24.400000 21.873494 16.548193
## [36] 0.500000 17.094937 0.500000 1.500000 18.954545 18.162651 25.545455
## [43] 2.375000 0.500000 12.928571 1.000000 19.102564 7.917910 21.977273
## [50] 0.500000 24.561728 0.500000 0.500000 18.333333 24.909639 17.383721
## [57] 0.500000 14.360465 0.500000 20.409091 19.071429 0.500000 25.712500
## [64] 16.189655 15.125000 15.949275 12.648148 1.000000 26.880952 25.594118
## [71] 19.435897 0.500000 21.331325 25.444444 21.790698 20.487805 7.993333
## [78] 14.849206 21.682927 17.750000 19.857143 0.500000 11.533333 20.556180
## [85] 0.500000 23.768293 21.006329 21.378378 0.500000 19.328947 13.620000
## [92] 10.661290 21.632530 14.030864 0.500000 1.000000 0.500000 26.466667
## [99] 28.084270 18.606061
```

```
#or
```

```
meanInfs2 <- sapply(sims, meanInfections)
```

`sapply` statements are essentially just `lapply` statements that simplify the result to a vector. This is useful when the output of the function is a single value, and not so useful when function returns multiple values.

A side note: some people will criticize `for` loops in R, and say “just use `apply`, it’s faster”. It’s not, really. Write however you feel comfortable. For awhile, `apply` statements were super confusing to me, so I tended to

use `for` loops instead. After more work in, I shifted and tend to use `apply` statements when it fits, as they are less code and are more intuitive to me for many situations.

Let's practice a bit.

Calculate the maximum number of infected individuals at any time in the `sims` list using the `apply` approach.

What is the mean duration (the total time the epidemic took before it stopped) across all the epidemics in `sims`?

plyr apply functionality tweaks

`XYply` statements as nice wrappers to more classic `apply` statements. Here, `X` and `Y` can take values of 'a', 'l', or 'd', depending on the input or output data structure desired. For instance, if we have a list that we would like to apply over and return a `data.frame`, we would use `ldply`, where the `l` is claiming that the input is a list object, and the `d` is claiming that the output should be formatted as a `data.frame`. Other examples of this syntax would be `adply`, `ddply`, `laply`, `aaply`, etc. etc.

Below, I provide an example of the `aXply` syntax (e.g., `adply`, `alply`, `aaply`).

```
arr <- array(1:27, c(3,3,3))
rownames(arr) = c("Curly", "Larry", "Moe")
colnames(arr) = c("Groucho", "Harpo", "Zeppo")
dimnames(arr)[[3]] = c("Bart", "Lisa", "Maggie")
```

arr

```
## , , Bart
##
##      Groucho Harpo Zeppo
## Curly      1      4      7
## Larry      2      5      8
## Moe        3      6      9
##
## , , Lisa
##
##      Groucho Harpo Zeppo
## Curly     10     13     16
## Larry     11     14     17
## Moe       12     15     18
##
## , , Maggie
##
##      Groucho Harpo Zeppo
## Curly     19     22     25
## Larry     20     23     26
## Moe       21     24     27
```

Arrays are something that we did not introduce when we talked about `R` basics, and that is because they really are not used *too* often. Think of matrix. It has two dimensions (x and y), so it can be viewed as a rectangle of data. Arrays simply add more dimensions. In the example above, there is another dimension, forming a data cube (in the rectangle analogy).

We can use `plyr` functionality to operate on this array and return different forms. For instance, `aaply` takes an array and returns a simplified array (here a vector).

```
plyr::aaply(arr, 1, sum)
```

```
## Curly Larry Moe
## 117 126 135
```

We can change one letter and now return a data.frame containing two columns. This is also a good time to point out the flexibility of the XYply statements to different margins. Margins (denoted as `.margins` argument in R, asks along which axis you would like to operate on the array). If we set `.margins=1`, this corresponds to a row-wise operation, so we calculate the sum across the array for Curly, Larry, and Moe. If we change this to `.margins=2`, we operate on columns, and will return sums for Groucho, Harpo, and Zeppo. And if we use `.margins=3`, we will return sums for Bart, Lisa, and Maggie.

```
plyr::adply(.data=arr, .margins=1, .fun=sum)
```

```
##      X1 V1
## 1 Curly 117
## 2 Larry 126
## 3 Moe 135
```

```
plyr::adply(.data=arr, .margins=2, .fun=sum)
```

```
##      X1 V1
## 1 Groucho 99
## 2 Harpo 126
## 3 Zeppo 153
```

```
plyr::adply(.data=arr, .margins=3, .fun=sum)
```

```
##      X1 V1
## 1 Bart 45
## 2 Lisa 126
## 3 Maggie 207
```

Finally, we can return a list object. In this use case, this is not super helpful, but in other use cases the list output is pretty helpful.

```
plyr::alply(.data=arr, .margins=1, .fun=sum)
```

```
## $`1`
## [1] 117
##
## $`2`
## [1] 126
##
## $`3`
## [1] 135
##
## attr("split_type")
## [1] "array"
## attr("split_labels")
##      X1
## 1 Curly
## 2 Larry
## 3 Moe
```

A pitch for `plyr::ldply`. I really like this function, as I often find myself with lists of similar structures that I want to operate on and get a single clean object back. I will not go into an example, but this is a pretty useful function (though all the utility is basically contained in `vapply`).

Finally, you may wonder why am I pushing apply statements so hard. It has nothing to do with speed, and only a bit to do with code clarity. The main advantage is understanding the programmatic nature

of apply statements (which will be similar but less chronological than a for loop), and many parallel computing packages have their own little versions of apply statements ready to go (e.g., `parallel::mclapply`, `parallel::parLapply`, `parallel::clusterApplyLB`).

Let's do one practice problem to showcase the utility of `ldply` specifically.

Calculate the correlation between number of infections and time for each simulation, reporting the estimate, p-value, and confidence intervals around the estimate. (you will use `cor.test` to do this, whose output is a list object as well)

A note about `do.call` and `Reduce`

While a bit opaque, these functions are pretty useful in a variety of situations. Speaking of data manipulation functions that are useful but a bit conceptually difficult, `do.call` and `Reduce` are solid base R functions.

`do.call` is a way of calling the same function recursively on multiple objects, and may have similar output to `Reduce`, which is also a way to recursively apply a function.

```
lst <- list(1:10, 1:10, 1:10, 1:10, 1:10)
lst

## [[1]]
## [1] 1 2 3 4 5 6 7 8 9 10
##
## [[2]]
## [1] 1 2 3 4 5 6 7 8 9 10
##
## [[3]]
## [1] 1 2 3 4 5 6 7 8 9 10
##
## [[4]]
## [1] 1 2 3 4 5 6 7 8 9 10
##
## [[5]]
## [1] 1 2 3 4 5 6 7 8 9 10
#this makes a single rbind call with each element of the list as an argument
str(do.call(rbind, lst))

## int [1:5, 1:10] 1 1 1 1 1 2 2 2 2 2 ...
#this does it iteratively (so makes n-1 rbind calls)
Reduce(rbind, lst)

##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## init    1    2    3    4    5    6    7    8    9    10
##         1    2    3    4    5    6    7    8    9    10
##         1    2    3    4    5    6    7    8    9    10
##         1    2    3    4    5    6    7    8    9    10
##         1    2    3    4    5    6    7    8    9    10
```

Practice problems with lists

```
set.seed(123)
mats <- lapply(1:100, function(x){
  matrix(rbinom(5000, 1, 0.3), ncol=rpois(1,20))
})
```


[illegible]

[illegible]

```
## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [334]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [313]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [239]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [313]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [239]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [186]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [417]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [186]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [334]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [313]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [228]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [334]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [218]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [193]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [228]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [264]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [295]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [186]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [278]

## Warning in matrix(rbinom(5000, 1, 0.3), ncol = rpois(1, 20)): data length
## [5000] is not a sub-multiple or multiple of the number of rows [358]
```

Use either a for loop or an apply statement to calculate the number of rows and columns in the `mats` list defined above.

Calculate the column sums of each matrix in `mats` and return the object as a list.

Subset `mats` to include only the matrices that have more than 250 rows. How many matrices are left in the subset list?

Calculate the fraction of values that are 1 for each matrix in `mats`

Write code to make each matrix column it's own list element, while keeping the structure of the `mats` list.

So this means that list element 1 would have 16 nested list elements, with each list element corresponding to the values of columns from `mats[[1]]`.

sessionInfo

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 22.04.5 LTS
##
## Matrix products: default
## BLAS: /usr/lib/x86_64-linux-gnu/atlas/libblas.so.3.10.3
## LAPACK: /usr/lib/x86_64-linux-gnu/atlas/liblapack.so.3.10.3; LAPACK version 3.10.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8     LC_NAME=C
##  [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/New_York
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] igraph_2.1.4  stringr_1.5.1
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.37    fastmap_1.2.0    xfun_0.52        magrittr_2.0.3
##  [5] glue_1.8.0       knitr_1.50       pkgconfig_2.0.3  htmltools_0.5.8.1
##  [9] rmarkdown_2.29   tinytex_0.57     lifecycle_1.0.4  cli_3.6.5
## [13] compiler_4.5.0   plyr_1.8.9       tools_4.5.0      evaluate_1.0.4
## [17] Rcpp_1.0.14      yaml_2.3.10      rlang_1.1.6      stringi_1.8.7
```